

Successfully implemented I2C communication between PIA and RTC in assembler.

 flexonsbd.blogspot.com/2018/10/piartci2c.html

Saturday, October 27, 2018

I successfully established I2C communication between the PIA and RTC using assembly language.

I was able to read the RTC time via PIA using assembly language.

The RTC module I used is the HiLetgo ZS-042, which I obtained from Amazon. This module uses a DS3231 chip and is designed to set and read the time via I2C communication. I only knew the name of I2C communication and didn't know anything specific about it, but since manually entering the date when starting FLEX is cumbersome, I decided to try automating the reading from the RTC.

However, most of the information I found online was for connecting with Arduino, and most of it used the Wire library. I also checked whether the RTC module would work with Arduino, and while the ease of getting it done in less than 30 minutes is very much appreciated (thanks to the person who created the library...), it's completely useless if you want to write an I2C communication program in 6809 assembler.

I also found a few examples of I2C communication using PIC and AVR microcontroller assemblers, but these mostly use microcontrollers that have I2C communication capabilities built in, so these are also useless. There were also cases with PIC and AVR microcontrollers that lack I2C communication capabilities, but these microcontrollers have port bit processing instructions, so they weren't very helpful as a reference for programming the 6809, which lacks bit processing instructions.

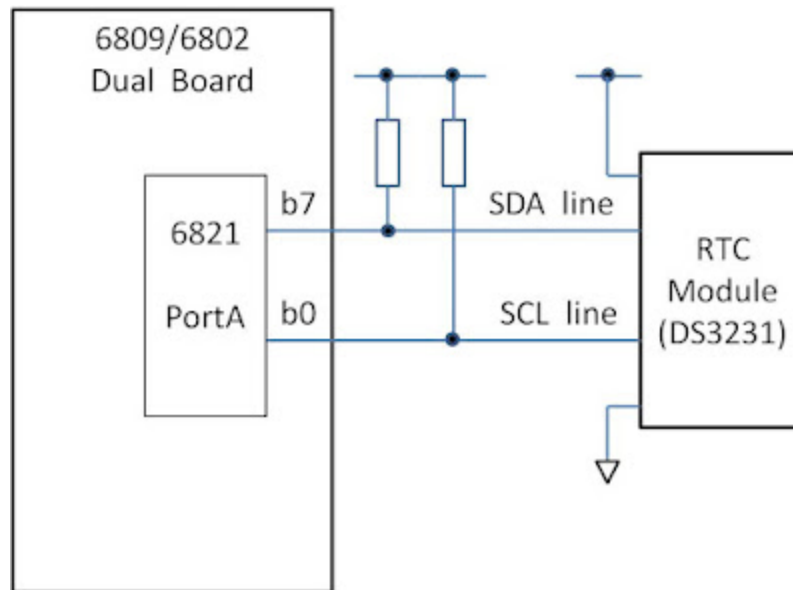
Therefore, in the end, I had to write the program myself by reading explanations of I2C communication and the DS3231 datasheet.

Connection between 6821 and RTC

The 6821 and RTC were wired as shown in the diagram below.

The pull-up resistor in the diagram is replaced with the built-in resistor of Port A.

The 6821's Port A is open-drain and is pulled up with a 5k Ω resistor, making it suitable for I2C communication. Furthermore, it does not require switching between input and output, and can be used as an input while remaining set to output (however, it is necessary to output a high value beforehand), which I thought would make programming easier.



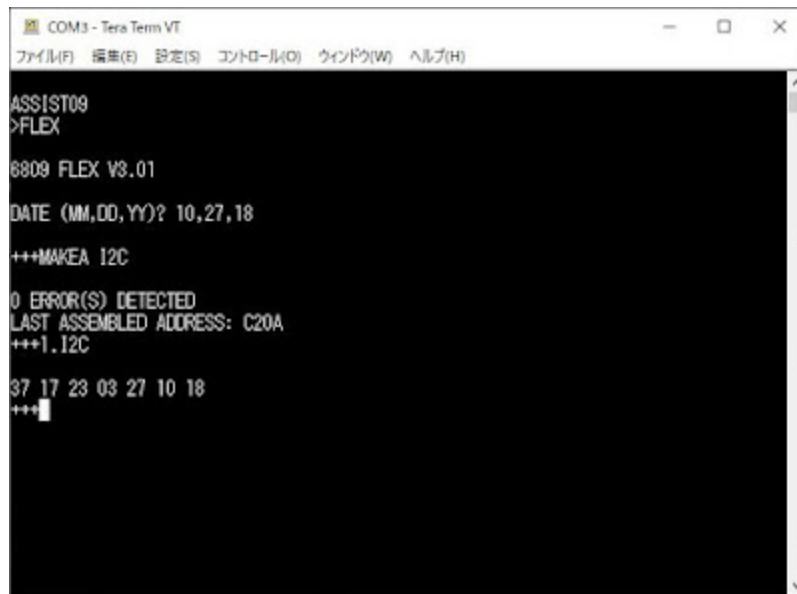
RTC Wiring Diagram

Since I don't own a logic analyzer, I continued to experiment while looking at the waveforms with a digital oscilloscope, but I couldn't even tell if I was succeeding from the very first step of writing the slave address, and I wasn't getting any ACKs at all. There were times when I almost gave up and thought I would have to insert a PIC microcontroller in between.

In the end, I realized that the SDA signal outputs 0 when low, but not 1 when high, but rather an open circuit, i.e., Hi-Z. By rewriting the code to correspond to this, I was finally able to read the data.

Read successful

The data shown in the diagram below is a reading of the first seven registers of the RTC, representing, in order, seconds, minutes, hours, day of the week, day, month, and year.



```
COM3 - Tera Term VT
ファイル(F) 編集(E) 設定(S) コントロール(C) ウィンドウ(W) ヘルプ(H)

ASSIST09
>FLEX
6809 FLEX V3.01
DATE (MM,DD,YY)? 10,27,18
+++MAKEA I2C
0 ERROR(S) DETECTED
LAST ASSEMBLED ADDRESS: C20A
+++1.I2C
37 17 23 03 27 10 18
+++
```

Read the date and time from the RTC.

At this stage, we've only just managed to get to the point where we can read data. All that's left is to create a writing routine, ~~format it as a resident program, and~~ replace the date input routine in the FLEX initialization routine.

(The program will be made public once it's finished.)

[Added November 4th] The completed program was published in the blog on November 3rd.